CYLIXLAB · ASSESSMENT

Module 04

Computer Vision & Visual Intelligence

MCQ Assessment | 30 technical questions with green-check marked answers and full explanations

Assessment overview

Part	Questions	Topic
A	Q1–Q5	Image processing: grayscale conversion, image memory, Sobel filtering, median filter, augmentation with boxes
B	Q6–Q10	CNN backbones: output size, conv parameters, receptive field, depthwise-separable convs, global average pooling
C	Q11–Q15	ResNet: degradation problem, residual gradients, bottleneck parameters, depth counting, projection shortcuts
D	Q16–Q20	Object detection: IoU, box formats, non-maximum suppression, focal loss, precision and recall
E	Q21–Q25	YOLO: output tensor, box decoding, multi-scale predictions, YOLOv1 limits, confidence threshold
F	Q26–Q30	Segmentation: task types, transposed convolution, Dice vs IoU, U-Net skips, dilated convolution

HOW TO USE THIS ASSESSMENT

1. Cover the answers and choose one option per question (about 90 seconds each).
2. Check your answer; the correct option is highlighted and marked with a green check.
3. Read **every** explanation, including for questions you got right: the “why the others fail” notes target common misconceptions.
4. Score yourself with the answer key on the last page and revisit the matching theory section.

A Image Processing

Q1 IMAGE PROCESSING

Using Y = 0.299R + 0.587G + 0.114B, what is the grayscale value of the pixel **(R, G, B) = (100, 200, 50)**?

- **A.** ≈ 116.7
- ✓ **B. ≈ 153.0**
- **C.** 200
- **D.** 350

Correct answer: B

Why it is correct. Y = 0.299·100 + 0.587·200 + 0.114·50 = 29.9 + 117.4 + 5.7 = **153.0**. The weights reflect human sensitivity: green contributes most to perceived brightness, blue least.

Why the others fail.

✗ **A.** 116.7 is the plain average (100 + 200 + 50) / 3, which ignores perceptual weighting.
✗ **C.** 200 is the maximum channel (the V channel of HSV), not luminance.
✗ **D.** 350 is the sum of the channels and exceeds the 0–255 range.

Q2 IMAGE PROCESSING

A **1280 × 720 RGB** image is converted to **float32** before being fed to a network. How many bytes does it occupy?

- **A.** 2,764,800
- **B.** 921,600
- **C. 11,059,200**
- **D.** 22,118,400

Correct answer: C
Why it is correct. Values = 1280 × 720 × 3 = 2,764,800. Each float32 value takes 4 bytes: 2,764,800 × 4 = **11,059,200 bytes ≈ 11 MB**. Converting from uint8 multiplies memory by 4.

Why the others fail.
✗ **A.** 2,764,800 bytes is the uint8 size (1 byte per value).
✗ **B.** 921,600 = 1280 × 720 counts a single channel at 1 byte.
✗ **D.** 22,118,400 uses 8 bytes per value (float64).

Q3 FILTERING

The Sobel-x kernel [−1 0 1; −2 0 2; −1 0 1] is applied (no flipping) to the 3 × 3 patch whose three rows are each **[0, 50, 100]**. What is the output value?

- **A.** 300
- **B.** 150
- **C.** 0
- **D. 400**

Correct answer: D
Why it is correct. In each row the kernel computes −1·0 + 0·50 + 1·100 = 100. The rows are weighted 1, 2, 1, so the result is 100 + 200 + 100 = **400**. A large value means a strong left-to-right intensity increase: a vertical edge.

Why the others fail.
✗ **A.** 300 forgets that the middle row has weight 2.
✗ **B.** 150 is the sum of one row, unrelated to the kernel.
✗ **C.** 0 is what Sobel-y gives here, because every column is constant from top to bottom.

Q4 FILTERING

A camera image contains random isolated black and white pixels (**salt-and-pepper noise**). Which filter removes them best while keeping edges sharp?

- ✓ **A. Median filter**
- **B.** Box (mean) filter
- **C.** Laplacian filter
- **D.** Sobel filter

Correct answer: A

Why it is correct. A median filter replaces each pixel with the middle value of its neighbourhood. A single extreme pixel never becomes the median, so it disappears, and because the median is an existing value rather than an average, edges are not smeared.

Why the others fail.

✗ **B.** A mean filter spreads each outlier into its neighbours and blurs edges.
✗ **C.** The Laplacian is a second-derivative edge detector and amplifies isolated spikes.
✗ **D.** Sobel measures gradients; it detects the noise rather than removing it.

Q5 AUGMENTATION

An image **640 px wide** has a box with $x_1 = 100$ and $x_2 = 200$. After a **horizontal flip**, what are the new x-coordinates of the box?

- **A.** $x_1 = 100$, $x_2 = 200$ (unchanged)
- **B.** $x_1 = 540$, $x_2 = 440$
- ✓ **C. $x_1 = 440$, $x_2 = 540$**
- **D.** $x_1 = -540$, $x_2 = -440$

Correct answer: C

Why it is correct. A horizontal flip maps $x \to W - x$. The old right edge becomes the new left edge: $x_1' = 640 - 200 = \mathbf{440}$, $x_2' = 640 - 100 = \mathbf{540}$. Geometric augmentation must always transform the labels too.

Why the others fail.

✗ **A.** The object has moved, so an unchanged box now labels the wrong region.
✗ **B.** These are the right numbers in the wrong order; x_1 must stay smaller than x_2.
✗ **D.** Negating coordinates without adding the width W puts the box outside the image.

B CNN Backbones

Q6 OUTPUT SIZE

A **128 × 128** feature map passes through a 3 × 3 convolution with **stride 2** and **padding 1**. What is the output height and width?

- **A.** 128
- **B.** 63
- **C.** 126
- ✓ **D. 64**

Correct answer: D
Why it is correct. $O = \lfloor (W - K + 2P)/S \rfloor + 1 = \lfloor (128 - 3 + 2)/2 \rfloor + 1 = \lfloor 63.5 \rfloor + 1 = \mathbf{64}$. A stride-2, "same"-padded 3 × 3 convolution halves the resolution, the standard downsampling step in modern backbones.

Why the others fail.
✗ **A.** 128 ignores the stride; it is the stride-1 result.
✗ **B.** 63 forgets the final + 1.
✗ **C.** 126 is the stride-1 result without padding (128 − 3 + 1).

Q7 PARAMETERS

How many trainable parameters does a convolution with **5 × 5** kernels, **32** input channels and **64** output channels (with bias) have?

- **A.** 51,200
- ✓ **B. 51,264**
- **C.** 1,664
- **D.** 102,528

Correct answer: B
Why it is correct. Each filter has 5·5·32 = 800 weights + 1 bias = 801. With 64 filters: 801 × 64 = **51,264**. The count does not depend on the image size, because the same weights are shared across all positions.

Why the others fail.
✗ **A.** 51,200 leaves out the 64 biases.
✗ **C.** 1,664 = (25 + 1)·64 ignores the 32 input channels; every filter spans all input channels.
✗ **D.** 102,528 doubles the correct count.

Q8 RECEPTIVE FIELD

Three **3 × 3** convolutions (stride 1) are stacked, each with C input and C output channels. Compared with one 7 × 7 convolution, what is true?

- ✓ **A. Same 7 × 7 receptive field, with $27C^2$ weights instead of $49C^2$ and three non-linearities instead of one**
- ○ **B.** A 9 × 9 receptive field, because the kernel sizes add up
- ○ **C.** The same 7 × 7 receptive field and the same number of weights
- ○ **D.** Only a 3 × 3 receptive field, because each kernel is 3 × 3

Correct answer: A

Why it is correct. Each stride-1 3 × 3 layer extends the receptive field by 2: 3 → 5 → 7. Weights: $3 \times 9C^2 = 27C^2$ versus $49C^2$, about 45% fewer, and the extra ReLUs make the function more expressive. This is VGG's design idea.

Why the others fail.

✗ **B.** Receptive fields grow by K − 1 per layer, not by K: 3 + 2 + 2 = 7.
✗ **C.** The receptive field matches, but the stack needs far fewer weights.
✗ **D.** Stacking enlarges the region each output can see; the deepest output sees 7 × 7 input pixels.

Q9 EFFICIENT CONVS

A **depthwise-separable** convolution replaces a 3 × 3 convolution from **64** to **128** channels (no biases). How many weights does it have?

- ○ **A.** 576
- ○ **B.** 8,192
- ✓ **C. 8,768**
- ○ **D.** 73,728

Correct answer: C

Why it is correct. Depthwise: one 3 × 3 filter per input channel = 9 × 64 = 576. Pointwise: a 1 × 1 conv from 64 to 128 = 8,192. Total = **8,768**, about 8.4× fewer than the standard 73,728. MobileNet is built on this idea.

Why the others fail.

✗ **A.** 576 is only the depthwise part, which cannot change the number of channels.
✗ **B.** 8,192 is only the pointwise part.
✗ **D.** 73,728 = 9·64·128 is the standard convolution being replaced.

Q10 POOLING

A backbone outputs a **7 × 7 × 512** feature map. **Global average pooling** is followed by a fully-connected layer with bias to **10** classes. How many parameters does the FC layer have?

- ✓ **A. 5,130**
- **B.** 250,890
- **C.** 5,120
- **D.** 490

Correct answer: A

Why it is correct. GAP averages each channel to one number, giving a 512-vector. FC: 512 × 10 weights + 10 biases = **5,130**. Flattening instead would give 25,088 inputs and 250,890 parameters, about 49× more and prone to overfitting.

Why the others fail.

✗ **B.** 250,890 is the count when the 7 × 7 × 512 map is flattened without pooling.
✗ **C.** 5,120 leaves out the 10 biases.
✗ **D.** 490 = 7 × 7 × 10 ignores the 512 channels entirely.

C ResNet

Q11 RESNET

A 56-layer plain CNN shows **higher training error** than a 20-layer plain CNN on the same data. What does this indicate?

- **A.** Overfitting, so the deeper model needs more dropout
- **B.** The dataset is too small for 20 layers
- ✓ **C. An optimization difficulty (the degradation problem): extra plain layers are hard to train, even toward identity**
- **D.** The learning rate was too low for the shallow network

Correct answer: C

Why it is correct. Overfitting would show *lower* training error with higher validation error. Here even training error is worse, so the optimizer cannot use the extra layers. Residual connections fix this by letting blocks learn F(x) ≈ 0, which makes identity easy.

Why the others fail.

✗ **A.** Overfitting means fitting the training data too well; this model fits it worse.
✗ **B.** The 20-layer model trains fine; the problem appears only in the deeper one.
✗ **D.** The shallow network is the one performing better, so its learning rate is not the issue.

Q12 RESIDUAL GRADIENT

In a scalar toy model, each of 10 stacked blocks has $\partial F/\partial x$ = **0.01**. With a residual connection y = F(x) + x, what factor scales the gradient through all 10 blocks?

- **A.** 10^{-20}
- **B. ≈ 1.105** ✓
- **C.** 0.1
- **D.** 10.1

Correct answer: B

Why it is correct. With the shortcut, $\partial y/\partial x = \partial F/\partial x + 1 = 1.01$ per block, so the product is $1.01^{10} \approx$ **1.105**. Without it, the factor would be $0.01^{10} = 10^{-20}$: a completely vanished gradient. The "+ 1" is the gradient highway.

Why the others fail.

✗ **A.** 10^{-20} is the plain network without shortcuts.
✗ **C.** 0.1 adds the derivatives (10 × 0.01); the chain rule multiplies them.
✗ **D.** 10.1 also adds instead of multiplying (10 × 1.01).

Q13 BOTTLENECK

A bottleneck block keeps **512** channels at its input and output and reduces to **128** inside (1 × 1 → 3 × 3 → 1 × 1, no biases). How many weights does it have?

- **A.** 4,718,592
- **B.** 147,456
- **C.** 131,072
- **D. 278,528** ✓

Correct answer: D

Why it is correct. 1 × 1 (512 → 128) = 65,536; 3 × 3 (128 → 128) = 147,456; 1 × 1 (128 → 512) = 65,536. Total = **278,528**, about 17× fewer than two plain 3 × 3 convolutions at 512 channels.

Why the others fail.

✗ **A.** 4,718,592 = $2 \cdot 9 \cdot 512^2$ is two plain 3 × 3 convs at full width, which the bottleneck replaces.
✗ **B.** 147,456 is only the middle 3 × 3 convolution.
✗ **C.** 131,072 is only the two 1 × 1 convolutions.

Q14 RESNET

ResNet-50 uses bottleneck blocks arranged **[3, 4, 6, 3]** across its four stages. Where does the number **50** come from?

- ✓ **A. 1 stem convolution + 16 blocks × 3 convolutions + 1 fully-connected layer = 50 weighted layers**
- ○ **B.** It contains 50 residual blocks
- ○ **C.** 16 blocks × 3 convolutions + 2 projection shortcuts = 50
- ○ **D.** The first layer has 50 filters

Correct answer: A
Why it is correct. 3 + 4 + 6 + 3 = 16 bottleneck blocks, each with 3 convolutions = 48. Adding the 7 × 7 stem convolution and the final FC layer gives **50**. Projection shortcuts, BN and pooling layers are not counted.

Why the others fail.
✗ **B.** ResNet-50 has 16 blocks, not 50.
✗ **C.** Projection shortcuts are not part of the conventional count, and there are 4 of them, not 2.
✗ **D.** The stem has 64 filters; the name refers to depth.

Q15 SHORTCUTS

At the start of a ResNet stage the block uses **stride 2** and doubles the channels. What does the shortcut do?

- ○ **A.** It is removed for that block
- ○ **B.** It stays an identity; the addition broadcasts automatically
- ✓ **C. It becomes a 1 × 1 convolution with stride 2 (plus BN) so x matches the shape of F(x)**
- ○ **D.** It uses max pooling only, keeping the channel count

Correct answer: C
Why it is correct. F(x) + x needs identical shapes. A strided 1 × 1 projection halves the spatial size and changes the channels to match. This is the only kind of shortcut with weights; all others remain free identities.

Why the others fail.
✗ **A.** Removing it would lose the residual benefit exactly where the gradient path matters.
✗ **B.** Tensors with different spatial sizes and channel counts cannot be added.
✗ **D.** Pooling fixes the spatial size but not the channel count.

D Object Detection

Q16 IOU

Box A = (0, 0, 10, 10) and box B = (5, 5, 15, 15) in (x_1, y_1, x_2, y_2) format. What is their IoU?

- **A.** 0.25
- **B.** 0.5
- **C. ≈ 0.143** ✓
- **D.** ≈ 0.167

Correct answer: C

Why it is correct. Intersection = (5, 5, 10, 10) → 5 × 5 = 25. Union = 100 + 100 − 25 = 175. IoU = 25 / 175 ≈ **0.143**. Even though each box overlaps a quarter of the other, IoU is low, so such a prediction would not count as a match at 0.5.

Why the others fail.

✗ **A.** 0.25 is intersection / area of A, not intersection / union.
✗ **B.** 0.5 is the shift as a fraction of the box size, not an area ratio.
✗ **D.** 0.167 = 25 / 150 subtracts the intersection twice from the summed areas.

Q17 BOX FORMATS

A COCO annotation gives a box as **(x, y, w, h) = (30, 40, 100, 50)**, where (x, y) is the top-left corner. What are its corners (x_1, y_1, x_2, y_2)?

- **A.** (30, 40, 100, 50)
- **B.** (−20, 15, 80, 65)
- **C.** (30, 40, 130, 140)
- **D. (30, 40, 130, 90)** ✓

Correct answer: D

Why it is correct. x_2 = x + w = 30 + 100 = 130 and y_2 = y + h = 40 + 50 = 90, so the box is **(30, 40, 130, 90)**. Mixing up formats is one of the most common causes of shifted boxes and near-zero mAP.

Why the others fail.

✗ **A.** Reads width and height as corner coordinates.
✗ **B.** Treats (30, 40) as the centre (YOLO convention) instead of the top-left corner.
✗ **C.** Adds the width to y as well, mixing up w and h.

Q18 NMS

Three boxes of the same class have scores 0.95, 0.90 and 0.60. IoU(0.95, 0.90) = 0.7, IoU(0.95, 0.60) = 0.3 and IoU(0.90, 0.60) = 0.8. With an NMS threshold of **0.5**, which boxes remain?

- **A.** Only 0.95
- **B.** 0.95 and 0.90
- **C. 0.95 and 0.60** ✓
- **D.** All three

Correct answer: C

Why it is correct. Keep 0.95. Remove 0.90, since its IoU with 0.95 is 0.7 > 0.5. Then 0.60 is compared only with the kept box: IoU 0.3 < 0.5, so it is **kept**. Result: **{0.95, 0.60}**. Suppressed boxes no longer suppress anything.

Why the others fail.

✗ **A.** Removes 0.60 because it overlaps 0.90, but 0.90 had already been suppressed.
✗ **B.** 0.90 overlaps the top box with IoU 0.7, above the threshold, so it must go.
✗ **D.** Without suppression the duplicate 0.90 detection of the same object would survive.

Q19 FOCAL LOSS

Focal loss multiplies cross-entropy by $(1 - p_t)^{\gamma}$. With $\boldsymbol{\gamma = 2}$, what factor is applied to an easy example with $\boldsymbol{p_t = 0.8}$?

- **A. 0.04** ✓
- **B.** 0.2
- **C.** 0.64
- **D.** 0.8

Correct answer: A

Why it is correct. $(1 - 0.8)^2 = 0.2^2 =$ **0.04**, so this easy example's loss is reduced 25×. A hard example with $p_t = 0.2$ keeps 0.64. Training therefore focuses on hard cases instead of the thousands of easy background anchors.

Why the others fail.

✗ **B.** 0.2 is $(1 - p_t)$ without the exponent γ.
✗ **C.** $0.64 = 0.8^2$, which squares p_t instead of $1 - p_t$.
✗ **D.** 0.8 is p_t itself, not the modulating factor.

Q20 METRICS

A detector makes **12** predictions, of which **9** correctly match objects. The image has **15** ground-truth objects. What are precision and recall?

- **A.** Precision 0.60, recall 0.75
- **B. Precision 0.75, recall 0.60**
- **C.** Precision 0.75, recall 0.75
- **D.** Precision 0.90, recall 0.60

Correct answer: B

Why it is correct. TP = 9, FP = 12 − 9 = 3, FN = 15 − 9 = 6. Precision = 9/12 = **0.75** (how many predictions are right). Recall = 9/15 = **0.60** (how many objects were found). Lowering the confidence threshold would usually raise recall and lower precision.

Why the others fail.

✗ **A.** Swaps precision and recall.
✗ **C.** Uses the prediction count as the denominator for both.
✗ **D.** 0.90 has no basis here; 9 of 12 predictions are correct, not 9 of 10.

E YOLO

Q21 OUTPUT TENSOR

A YOLOv1-style detector uses **S = 7**, **B = 2** boxes per cell and **C = 10** classes. What is the size of the output tensor?

- **A. 7 × 7 × 20 = 980 values**
- **B.** 7 × 7 × 30 = 1,470 values
- **C.** 7 × 7 × 15 = 735 values
- **D.** 7 × 7 × 12 = 588 values

Correct answer: A

Why it is correct. Each cell outputs B·5 + C = 2·5 + 10 = 20 numbers (x, y, w, h, confidence for each box plus one set of class probabilities), so the tensor is 7 × 7 × 20 = **980** values.

Why the others fail.

✗ **B.** 30 per cell is the Pascal VOC setting with C = 20, not C = 10.
✗ **C.** 15 = 5 + 10 counts only one box per cell.
✗ **D.** 12 = B + C forgets that each box needs 5 values.

Q22 BOX DECODING

A YOLOv2-style head at stride 32 predicts for cell $c_x = 5$ with $\sigma(t_x) = 0.25$, anchor width $p_w = 3$ cells and $t_w = \ln 2$. What are the box centre b_x and width b_w in pixels?

- **A.** $b_x = 160$ px, $b_w = 192$ px
- **B.** $b_x = 5.25$ px, $b_w = 6$ px
- **C.** $b_x = 176$ px, $b_w = 96$ px
- ✓ **D. $b_x = 168$ px, $b_w = 192$ px**

Correct answer: D

Why it is correct. $b_x = \sigma(t_x) + c_x = 0.25 + 5 = 5.25$ cells → $5.25 \times 32 =$ **168 px**. $b_w = p_w \cdot e^{t_w} = 3 \times 2 = 6$ cells → **192 px**.

Why the others fail.

✗ **A.** 160 px uses only the cell index and drops the 0.25 offset.
✗ **B.** These are the right values in grid units; they were not multiplied by the stride.
✗ **C.** 176 px assumes the cell centre (offset 0.5), and 96 px is the anchor width alone, ignoring e^{t_w}.

Q23 MULTI-SCALE

A YOLOv3-style model receives a **320 × 320** image and predicts at strides 32, 16 and 8 with **3 anchors** per cell. How many candidate boxes does it output?

- **A.** 2,100
- ✓ **B. 6,300**
- **C.** 10,647
- **D.** 4,800

Correct answer: B

Why it is correct. Grids: 320/32 = 10, 320/16 = 20, 320/8 = 40 → 100 + 400 + 1,600 = 2,100 cells. × 3 anchors = **6,300** boxes, which confidence filtering and NMS then reduce to a handful.

Why the others fail.

✗ **A.** 2,100 counts cells and forgets the 3 anchors per cell.
✗ **C.** 10,647 is the count for a 416 × 416 input.
✗ **D.** 4,800 = 1,600 × 3 counts only the finest scale.

Q24 YOLOv1

Why does the original YOLOv1 struggle with **flocks of small birds** close together?

- ✓ **A. Each grid cell predicts only a few boxes and a single set of class probabilities, so several small objects in one cell compete**
- **B.** It cannot process colour images
- **C.** Its NMS step is too slow for many objects
- **D.** It uses a two-stage region-proposal pipeline

Correct answer: A

Why it is correct. In a 7 × 7 grid each cell is responsible for objects whose centres fall inside it, with at most 2 boxes and one class distribution. Many small objects in one cell exceed that capacity. Later versions added anchors, finer grids and multi-scale heads.

Why the others fail.

✗ **B.** YOLO takes RGB input like other CNNs.
✗ **C.** NMS speed is not the bottleneck; the limit is how many objects each cell can represent.
✗ **D.** YOLO is a one-stage detector; proposals belong to Faster R-CNN.

Q25 INFERENCE

A deployed YOLO model's confidence threshold is raised from **0.25 to 0.80**. What is the most likely effect?

- **A.** Both precision and recall increase
- **B.** The model runs twice as fast because NMS is skipped
- **C.** mAP increases automatically
- ✓ **D. Precision usually rises and recall falls: fewer false positives, but more missed objects**

Correct answer: D

Why it is correct. A higher threshold keeps only very confident boxes. Most remaining boxes are correct (higher precision), but real objects with moderate scores are discarded (lower recall). The threshold should be tuned for the application's cost of misses versus false alarms.

Why the others fail.

✗ **A.** Precision and recall trade off when only the threshold changes.
✗ **B.** NMS still runs; it only processes fewer boxes.
✗ **C.** mAP is computed over all thresholds, so a deployment threshold does not change it.

F Image Segmentation

Q26 SEGMENTATION

A traffic system must outline **each car separately** in a street image so they can be counted. Which task is required?

- **A.** Semantic segmentation
- **B. Instance segmentation** ✓
- **C.** Image classification
- **D.** Edge detection

Correct answer: B

Why it is correct. Instance segmentation produces a separate mask per object, so two touching cars get different IDs and can be counted. Panoptic segmentation would also work, adding labels for background "stuff" such as road and sky.

Why the others fail.

✗ **A.** Semantic segmentation marks all car pixels with one label, merging touching cars.
✗ **C.** Classification gives one label for the whole image, with no location.
✗ **D.** Edges are not object masks and do not separate instances reliably.

Q27 UPSAMPLING

A transposed convolution with input size **8**, stride **2**, padding **1** and kernel **4** is applied. Using $O = (I - 1)S - 2P + K$, what is the output size?

- **A. 16** ✓
- **B.** 14
- **C.** 18
- **D.** 17

Correct answer: A

Why it is correct. $O = (8 - 1)\cdot 2 - 2\cdot 1 + 4 = 14 - 2 + 4 = \mathbf{16}$: exact 2× upsampling. This kernel-4, stride-2, padding-1 setting is the common choice in segmentation decoders and GANs.

Why the others fail.

✗ **B.** $14 = (I - 1)\cdot S$ stops before adding the kernel and subtracting padding.
✗ **C.** 18 forgets the $-2P$ term.
✗ **D.** 17 adds the "+ 1" from the ordinary convolution formula, which does not belong here.

Q28 METRICS

A predicted mask has **80** pixels, the ground-truth mask has **100** pixels and they overlap on **60** pixels. What is the **Dice** coefficient?

- **A.** 0.50
- **B.** 0.60
- **C. ≈ 0.667** ✓
- **D.** 0.75

Correct answer: C

Why it is correct. Dice = 2|A ∩ B| / (|A| + |B|) = 120 / 180 ≈ **0.667**. IoU here is 60 / (80 + 100 − 60) = 0.5, and indeed Dice = 2·0.5 / 1.5 = 0.667. Dice is always at least as large as IoU.

Why the others fail.

✗ **A.** 0.50 is the IoU, not Dice.
✗ **B.** 0.60 = 60/100 is the recall of the mask.
✗ **D.** 0.75 = 60/80 is the precision of the mask.

Q29 U-NET

What is the main purpose of the **skip connections** in U-Net?

- **A.** To reduce the number of parameters
- **B.** To replace the pooling layers
- **C.** To add residual identity mappings inside each conv block
- **D. To concatenate high-resolution encoder features with decoder features, restoring fine spatial detail lost by pooling** ✓

Correct answer: D

Why it is correct. Pooling discards precise locations. Copying encoder feature maps across and concatenating them with the upsampled decoder maps at the same scale lets the decoder draw sharp, well-placed boundaries, which is vital for small structures in medical images.

Why the others fail.

✗ **A.** Concatenation increases channels in the decoder, so parameters go up, not down.
✗ **B.** U-Net still uses max pooling in its encoder.
✗ **C.** U-Net skips concatenate across the network; they are not ResNet-style additions inside blocks.

Q30 DILATED CONV

DeepLab applies a **3 × 3** convolution with dilation rate **d = 3**. What region does it cover, and how many weights does each filter-channel have?

- **A.** 5 × 5 region, 9 weights
- **B. 7 × 7 region, 9 weights**
- **C.** 7 × 7 region, 49 weights
- **D.** 9 × 9 region, 9 weights

Correct answer: B

Why it is correct. $K_{eff} = K + (K - 1)(d - 1) = 3 + 2 \cdot 2 = \mathbf{7}$, while the kernel still has only **9** taps spaced 3 pixels apart. Dilation enlarges the receptive field without extra parameters or loss of resolution.

Why the others fail.

✗ **A.** 5 × 5 corresponds to d = 2.

✗ **C.** The taps are spread out; there are still only 9 of them, not 49.

✗ **D.** 9 × 9 multiplies K by d, which overstates the span by one tap gap.

✓ Answer Key & Score Sheet

Q	Ans	Topic	Q	Ans	Topic
1	**B**	Image Processing	16	**C**	Object Detection
2	**C**	Image Processing	17	**D**	Object Detection
3	**D**	Image Processing	18	**C**	Object Detection
4	**A**	Image Processing	19	**A**	Object Detection
5	**C**	Image Processing	20	**B**	Object Detection
6	**D**	CNN Backbones	21	**A**	YOLO
7	**B**	CNN Backbones	22	**D**	YOLO
8	**A**	CNN Backbones	23	**B**	YOLO
9	**C**	CNN Backbones	24	**A**	YOLO
10	**A**	CNN Backbones	25	**D**	YOLO
11	**C**	ResNet	26	**B**	Image Segmentation
12	**B**	ResNet	27	**A**	Image Segmentation
13	**D**	ResNet	28	**C**	Image Segmentation
14	**A**	ResNet	29	**D**	Image Segmentation
15	**C**	ResNet	30	**B**	Image Segmentation

Score interpretation

Score	Level	Recommended action
26–30	Mastery	Move on to Module 05; try explaining each answer aloud.
20–25	Proficient	Review the sections behind your misses, then redo those questions.
13–19	Developing	Re-read the Theory Study Guide sections, work the numeric examples by hand, retake after 48 hours.
0–12	Foundation	Restart with Sections 1 and 2 of the guide, then attempt the quiz again.

KEY IDEA
Track your misses by topic. If you missed 2+ questions in one part (A–F), that part is your priority for revision. Numeric questions (Q1, Q2, Q3, Q5, Q6, Q7, Q9, Q10, Q12, Q13, Q16, Q17, Q19, Q20, Q21, Q22, Q23, Q27, Q28, Q30) can all be solved with the formula sheet in Section 7 of the Module 04 Study Guide.